University Institute of Engineering
Department of Computer Science & Engineering

EXPERIMENT: 4

NAME: AAYUSH AMAN **UID: 23BCS14046**
BRANCH: BE-CSE **SECTION/GROUP: KRG_1B**
SEMESTER: 6TH **SUBJECT CODE: 23CSH-314**
SUBJECT NAME: SYSTEM DESIGN

1. Aim:

To design a scalable OTT (Over-The-Top) platform similar to Netflix or Amazon Prime that allows users to register, subscribe to plans, search and stream movies and TV shows in multiple resolutions, and receive personalized recommendations by identifying functional and non-functional requirements and defining system APIs.

2. Objective:

- Understand the architecture of a large-scale OTT video streaming platform.
- Identify functional requirements such as user onboarding, subscriptions, search, and video playback.
- Identify non-functional requirements including scalability, availability, and low latency streaming.
- Analyze CAP theorem trade-offs in OTT platforms.
- Design RESTful APIs for subscription management and video streaming.

3. Tools Used: Draw.io, Lucidchart, Excalidraw

4. Requirements Specification:

4.1. Functional Requirements:

- Clients should be able to create an account on the OTT platform.
- After successful login, clients should be able to choose and manage subscription plans.
- Clients should be able to search for movies or TV shows by title or name.
- Clients should be able to stream videos in multiple resolutions (480p, 720p, 1080p, 4K).
- Clients should receive recommendations for movies and TV shows (Optional).

4.2. Non-Functional Requirements:

- **Scalability:**
  - 200–300 million active users
  - Around 20,000 videos

    - Support for millions of concurrent streams
    - Horizontal scaling of services
- **Availability vs Consistency:**
    - Availability is more important than consistency
    - Video playback must not stop
    - Slight metadata delay is acceptable
    - Payment service requires strong consistency
- **Latency:**
    - Target latency: 50–80ms
    - Minimal buffering
    - CDN reduces delay

4.3. Core entities of the system:

- Client
- Subscription
- Video
- Video Metadata (Thumbnails, Description)
- Recommendation Engine

5. API Design:

- User Registration API: POST /api/users/register
- User Login API: POST /api/users/login
- Create Post API: POST /api/posts
- Follow User API: POST /api/users/{user_id}/follow
- Like Post API: POST /api/posts/{post_id}/like
- Comment API: POST /api/posts/{post_id}/comment
- Get Feed API: GET /api/feed?limit=20

6. High Level Design (HLD):

The system follows a microservices architecture.
Components:

1. API Gateway
- Handles authentication
- Routes requests
- Rate limiting

2. User Service
- Registration and login
- JWT authentication
- Uses MySQL

3. Subscription & Payment Service
- Manages plans
- Ensures transactional consistency

4. Search Service

- Uses ElasticSearch

5. Video Metadata Service

- Stores title, description, thumbnails (NoSQL DB)

6. Video Processing Pipeline

- Videos stored in Blob Storage
- Chunker splits videos into small segments
- Encoder creates multiple resolutions
- Kafka manages asynchronous processing

7. Streaming Service

- Generates HLS (.m3u8) or DASH (.mpd) files
- Client fetches chunks dynamically

8. CDN

- Caches video chunks
- Reduces server load
- Improves performance

7. Learning Outcomes:

1. Learned real-world OTT architecture design.
2. Understood functional vs non-functional requirements.
3. Gained knowledge of HLS/DASH and adaptive streaming.
4. Understood CAP theorem trade-offs.
5. Learned the role of Kafka, CDN, and encoding pipeline.
6. Improved understanding of scalable distributed systems